CSE406 – Cyber Security Sessional

Final Project Report

Model Extraction — Stealing a Classifier via a Black-Box HTTP Prediction API

Submitted by

Shatabdi Dutta Chowdhury 2105123
Tasnimzaman Tanmi 2105147

Supervised by

Dr. Md. Shariful Islam Bhuyan



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Submit Date : September 18, 2026

1 Introduction and Attack Definition

Many companies do not sell their trained machine-learning models. Instead, they put a model behind a web API: a user sends an input and gets back a prediction (a class label, and often the confidence scores), while the model itself stays hidden. A **model extraction attack** abuses this query interface to build a local copy of the hidden model, without ever seeing its weights or training data. This threatens the confidentiality of the model: the attacker can avoid paying per query, and can use the stolen copy to prepare further attacks.

This is **Project No. 33**. We implement and evaluate model extraction end-to-end against a real, network-reachable victim (not just an in-process function call), and we also add API-layer defenses. The work is based on three papers: Tramèr et al. (equation-solving attack) [1], Orekondy et al. (knockoff-net attack) [2], and Jagielski et al. (the accuracy-vs-fidelity evaluation framework) [3]. In our threat model the attacker is a separate process on a separate PC and talks to the victim *only* over HTTP; it never imports the victim's code or reads its weights.

2 Experimental Setup

2.1 System / Network Topology

The victim runs two Flask HTTP servers (undefended on port 5000, defended on port 5001). The attacker is a separate process on a separate PC that sends images as JSON and reads the JSON predictions back. It reaches the victim through a thin HTTP client (`RemoteOracle`) and never touches victim code. The only channel between the two sides is the HTTP request/response pair.

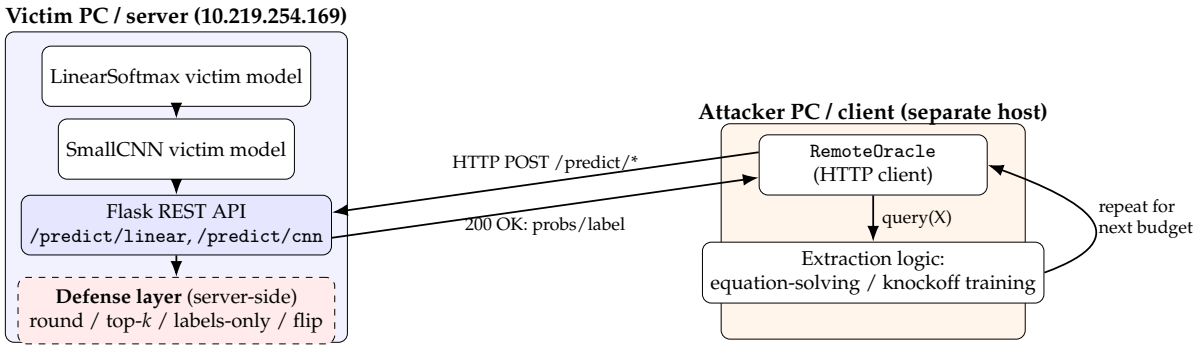


Figure 1: System topology. The attacker and victim are two physically separate PCs, connected only by an HTTP/TCP channel carried over a shared mobile-hotspot LAN. The attacker never has access to the victim's weights, training data, or source code — its only interface is the JSON request/response pair shown in Section 6.3.

2.2 Deployment Mode

The demo runs on **two physically separate PCs**. The victim binds both of its Flask servers to `0.0.0.0` so that the attacker PC can reach them over the shared hotspot LAN, and the attacker points its two URLs at the victim's current IPv4 address.

Role	Host	Ports / direction
Victim (server)	`10.219.254.169`	listens on TCP 5000 and 5001
Attacker (client)	separate PC, same hotspot LAN	outbound HTTP POST to both ports

Table 1: The deployment used for the demo: two separate PCs on one mobile-hotspot LAN. Because the hotspot assigns addresses by DHCP, the victim's IP can change between sessions; the attacker simply re-points `VICTIM_UNDEFENDED_URL` and `VICTIM_DEFENDED_URL` at the victim's current IP, read with `ipconfig` on the victim before each run.

2.3 Models, Data, and Reproducibility

Both victims were trained on real MNIST digits (a subset, to keep CPU training fast). All seeds are fixed. Small run-to-run changes of a few tenths of a percent are expected because of CPU/library nondeterminism.

Victim	Architecture	Train/Test	Seed	Clean acc.
LinearSoftmax	784→10 (softmax)	8000 / 2000	42	90.10%
SmallCNN	2 conv + 2 FC	8000 / 2000	42	96.50%

Table 2: Victim models trained on real MNIST.

2.4 Steps to Run the Demo

```
# ---- Victim PC (server) ----
python victim/victim_api.py # undefended API, port 5000
python victim/victim_api_defended.py none # defended API, port 5001
# both bind to 0.0.0.0 so the attacker PC can reach them

# ---- Attacker PC (client) ----
# point the attacker at the victim's current IP (from ipconfig on the victim;
# it can change because the two PCs are on a mobile hotspot)
$env:VICTIM_UNDEFENDED_URL = "http://10.219.254.169:5000"
$env:VICTIM_DEFENDED_URL = "http://10.219.254.169:5001"

python attacker/demo_query.py
python attacker/run_experiment.py
python attacker/defense_analysis.py
python attacker/arch_transfer_demo.py
python attacker/open_world_demo.py
python attacker/flip_determinism_demo.py
```

3 Attack 1 — Equation-Solving

3.1 Attack Process

The LinearSoftmax victim computes $f(x) = \text{softmax}(Wx + b)$, which is log-linear in the input. The attack recovers W and b with plain linear algebra, with no training:

1. Generate $d + 1 = 785$ random query images (any pixels work; digit shape is not needed).

2. Send them in one batch to the black-box linear victim and read the probability vectors.

3. For each class c, take the log-odds $\log\big(f_c(x)/f_0(x)\big) = (w_c - w_0)\cdot x + (b_c - b_0)$, which is linear in the unknowns.

4. Stack these into a linear system and solve it with least squares (`numpy.linalg.lstsq`).

5. Recover $\hat{W}$ and $\hat{b}$; no gradient descent is used.

3.2 Expected Outcome

Because the victim is truly log-linear, $d + 1 = 785$ independent queries fully determine the system. We therefore expected *near-perfect fidelity* ($\approx 100\%$) at 785 queries when the API returns full confidence scores, matching the exact extraction reported by Tramèr et al. Under any defense that perturbs the confidence vector we expected the attack to fall to roughly the 10% chance level and to *stay* there, since the linear relationship the attack depends on would no longer hold.

3.3 Actual Outcome

Scenario	Budget	Accuracy	Fidelity
No defense	785	90.10%	100.00%
Defense: round(2)	785	8.85%	9.55%
Defense: top-k(3)	785	5.95%	6.20%
Defense: labels-only	785	8.75%	8.40%
Defense: flip(0.3)	785	11.85%	11.85%

Table 3: Equation-solving: accuracy/fidelity versus defenses (at the minimum budget of 785 queries).

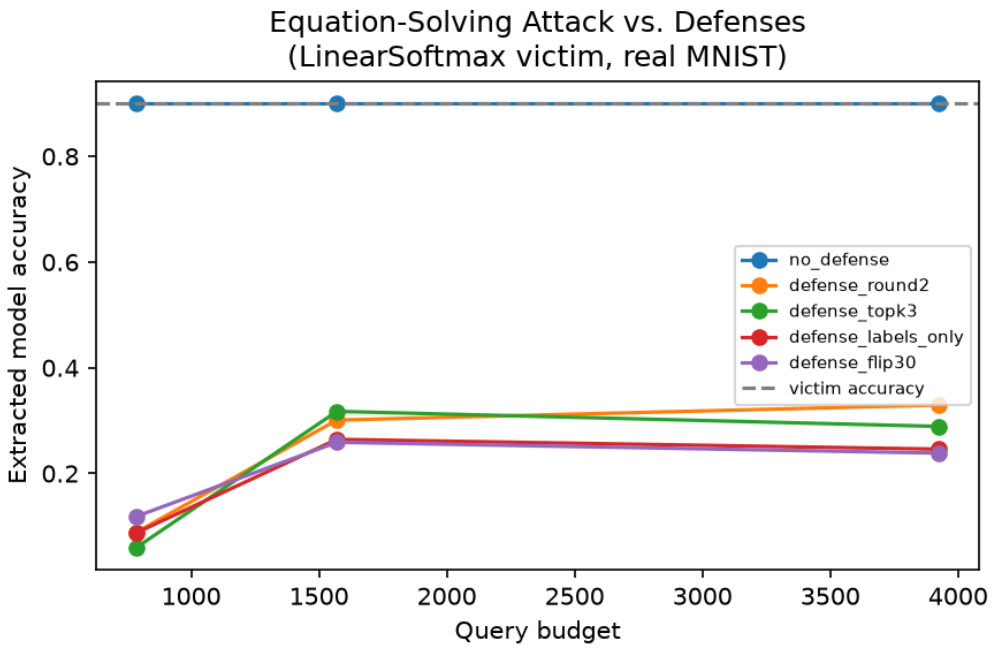


Figure 2: Extracted-model accuracy versus query budget for equation-solving, with no defense and under each of the four API-layer defenses.

3.4 Discrepancy (Actual vs. Expected) and Why

The headline result matched the prediction exactly: with no defense we got **100% fidelity** at exactly 785 queries, and the extracted accuracy equals the victim's own accuracy (90.1%) because a faithful copy also copies the victim's mistakes. At the minimum budget every defense also behaved as expected, holding the attack at 6–12% fidelity (Table 3). This is what the theory predicts: the attack works in log-odds space, so even a tiny change to the probabilities (for example rounding to 2 decimals) breaks the exact linear relationship and makes the system unsolvable.

One genuine discrepancy did appear *beyond* the minimum budget. We had expected the defended curves to stay flat at the chance level, but they do not: if the attacker keeps querying, the weaker defenses partially leak.

Defense	**B = 785**	**B = 1570**	**B = 3925**
round(2 decimals)	9.55%	31.50%	34.90%
top-3 truncation	6.20%	32.65%	29.75%
labels-only	8.40%	27.30%	24.95%
flip($\epsilon = 0.3$)	11.85%	20.70%	18.90%

Table 4: Equation-solving *fidelity* against each defense as the query budget grows past the minimum. The defended curves are not flat: they rise once the system is over-determined, then plateau far below 100%.

The reason is over-determination. At $B = 785$ the linear system has exactly as many equations as unknowns, so the perturbation introduced by the defense is absorbed directly into the solution and the result is essentially noise. At $B = 1570$ or 3925 there are two to five times more equations than unknowns, and least squares now averages the per-query perturbation instead of inheriting it. Rounding and truncation are *deterministic, zero-mean-ish* distortions of the probability vector, so averaging recovers a coarse approximation of the true decision boundary — enough for about 25–35% fidelity, but the information destroyed by the defense is gone for good, so the curve plateaus and never approaches the undefended 100%. Two further details support this reading: top-3 and labels-only, which destroy strictly more information, plateau lower than round(2); and `flip`, whose perturbation is a label change rather than a small numeric one, plateaus lowest of all (18.9%). The practical lesson is that confidence-hiding defenses degrade equation-solving rather than eliminating it, and a report that only quoted the 785-query column would have overstated how strong they are.

4 Attack 2 — Knockoff-Net

4.1 Attack Process

The CNN victim is non-linear, so equation-solving does not apply. Instead we use the victim as a teacher:

1. Build a transfer set of B query images.
2. Send them to the victim and keep only the top-1 predicted label (a pseudo-label).
3. Train a fresh substitute network from scratch on the (image, pseudo-label) pairs.
4. Evaluate the substitute for accuracy (vs. true labels) and fidelity (vs. the victim's predictions).

5. Repeat for budgets $B \in \{100, 300, 800, 2000, 4000\}$.

We also try three transfer distributions: *closed* (real MNIST from a split disjoint from the victim's), *open/noise* (random-noise images), and *open/Fashion* (Fashion-MNIST, real but out-of-distribution images).

4.2 Expected Outcome

Closed-world accuracy should rise with the query budget and approach the victim's accuracy. Open-world transfer with unrelated data should stay much lower; pure noise should stay near the 10% chance level.

4.3 Actual Outcome (Closed vs. Open-World)

Budget	**100**	**300**	**800**	**2000**	**4000**
Closed (acc.)	19.1%	66.8%	85.4%	91.7%	93.9%
Open / noise (acc.)	10.8%	10.8%	10.8%	10.8%	10.8%
Open / Fashion (acc.)	10.8%	12.3%	22.3%	34.9%	54.9%

Table 5: Knockoff accuracy versus budget across transfer distributions.

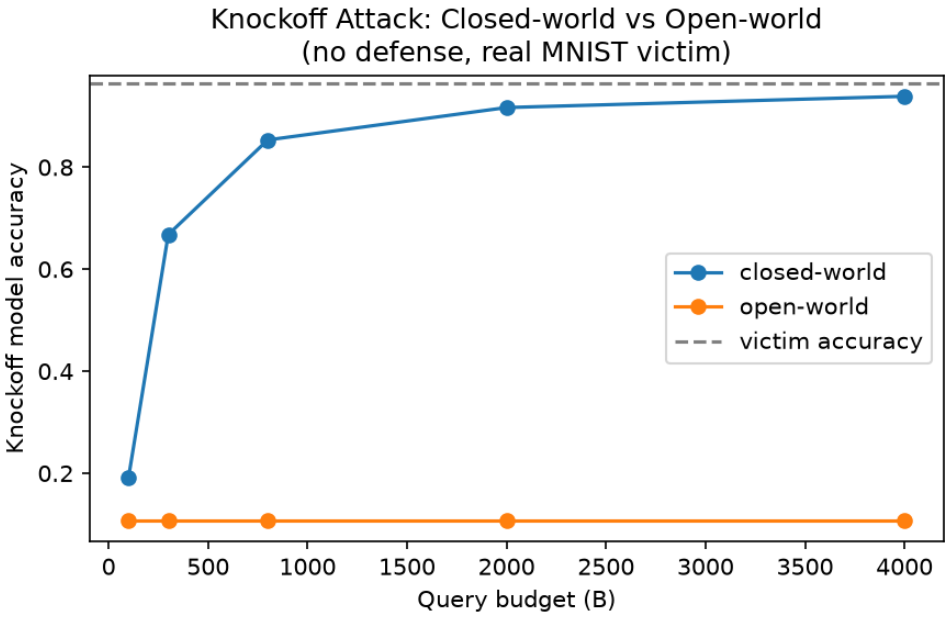


Figure 3: Closed-world versus noise-based open-world extraction.

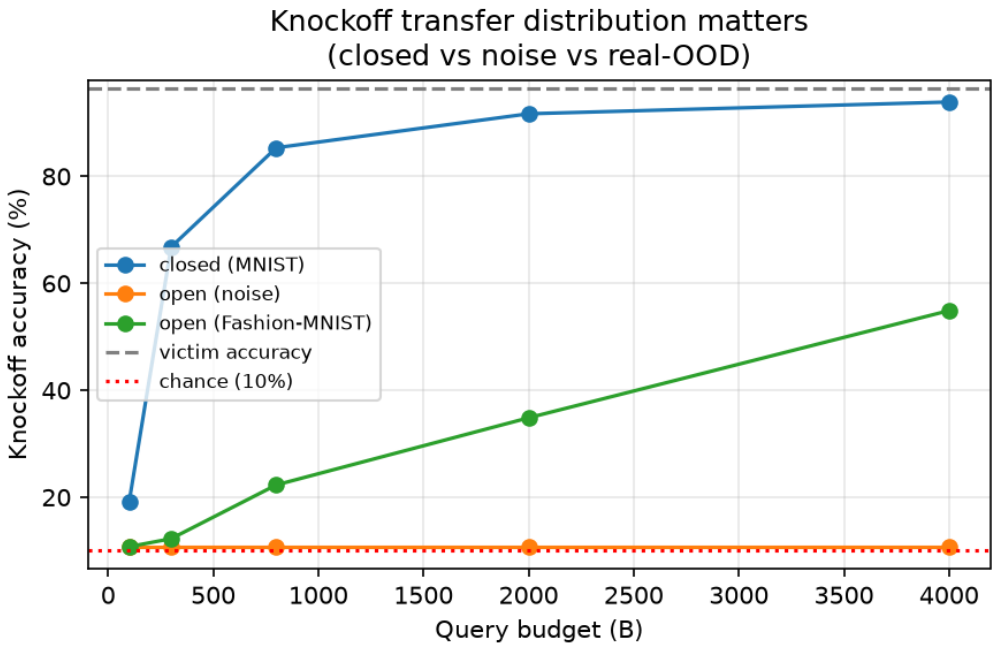


Figure 4: Transfer-distribution realism and extraction success: MNIST, random noise, and Fashion-MNIST queries.

4.4 Architecture-Transfer Robustness

To show the attack does not depend on knowing the victim's architecture, we also run it with a plain fully-connected MLP substitute (no convolutions) against the CNN victim.

Budget	**300**	**800**	**2000**	**4000**
CNN substitute (acc.)	67.4%	87.7%	93.1%	94.1%
MLP substitute (acc.)	79.7%	87.8%	89.8%	91.0%

Table 6: Architecture-independent stealing: CNN victim versus CNN and MLP substitutes.

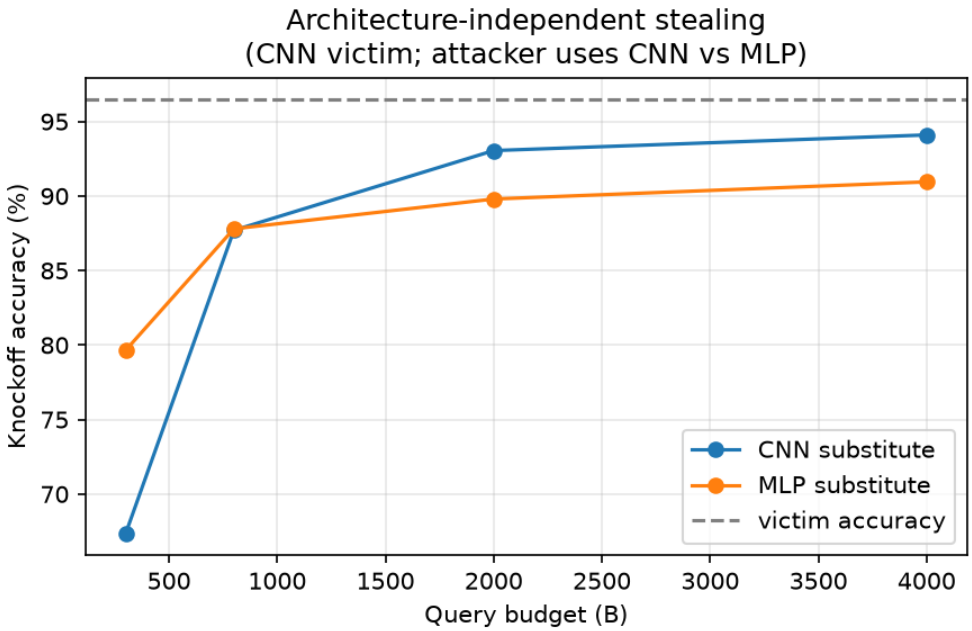


Figure 5: Architecture-independent extraction using CNN and MLP substitutes.

4.5 Discrepancy (Actual vs. Expected) and Why

The results matched expectations. Closed-world rose smoothly to 93.9% at 4000 queries (about 0.97× the victim). Open/noise stayed flat at the 10.8% chance level: random noise carries no useful signal, so the substitute can only guess the majority class. Open/Fashion is the interesting middle case: real but different images beat the noise floor and reach 54.9%, which shows that the *realism* of the query images, not just their number, is what drives the attack. The MLP substitute (a different architecture) still reached 91.0%, so the attack does not need the victim's architecture.

The only mild surprise was the direction of the CNN-vs-MLP gap at small budgets: at $B = 300$ the MLP substitute is *better* (79.7% vs. 67.4%), and the CNN only overtakes it from $B = 2000$ onwards. This is not a contradiction of Orekondy et al.; it is the usual sample-efficiency effect. With 300 labelled examples the smaller-capacity, fully-connected model fits the pseudo-labels with less variance, whereas the convolutional substitute needs more data before its inductive bias pays off. It reinforces the paper's claim rather than weakening it: the attacker does not even need the *right* architecture, and at a low budget a mismatched one can be the better choice.

5 Is the Attack Successful?

Attack 1 (equation-solving): successful, and exactly so. With no defense it reproduces the victim on 100% of test images using only 785 queries and no training. It succeeds because the victim is genuinely log-linear, so extraction reduces to solving a fully-determined linear system — there is no optimisation to get stuck in and no data to guess. It is defeated (though, as Table 4 shows, only degraded rather than eliminated) whenever the API hides or perturbs confidence scores, because the attack consumes the exact probability vector and nothing else.

Attack 2 (knockoff): successful in the closed-world. It reaches 93.9% accuracy and 95.8% fidelity with no knowledge of the victim's architecture, training data, or class meanings — only top-1 labels from the API. It succeeds because the victim itself supplies the supervision:

every query converts one unlabelled image the attacker already had into a labelled training example. It fails when the query distribution carries no task signal: pure noise stays at the 10.8% chance level, since the victim's answers on noise say nothing about its behaviour on digits. Confidence-hiding defenses do not stop it at all, because it only reads the argmax.

6 Observed Output Across PCs

Our deployment has exactly two hosts, and they play the two classical roles: the **victim PC is the server** — it runs the two Flask prediction APIs and listens on TCP 5000 and 5001 — and the **attacker PC is the client** — it opens the TCP connections and issues the HTTP POSTs. There is no third machine, proxy or broker anywhere in the path, so the three observations below (the victim's screen, the attacker's screen, and the request/response pair that crosses the wire between them) cover every PC involved in the attack.

6.1 Victim PC (server)

On the victim PC both servers are started in separate terminals and then left running for the whole demo. Each server prints `Running on http://0.0.0.0:5000` (respectively `:5001`) at start-up, which confirms it is bound to all interfaces and therefore reachable from the other PC, and thereafter Werkzeug logs one access line per incoming request showing the *attacker's* IP address, the endpoint, and the status code.

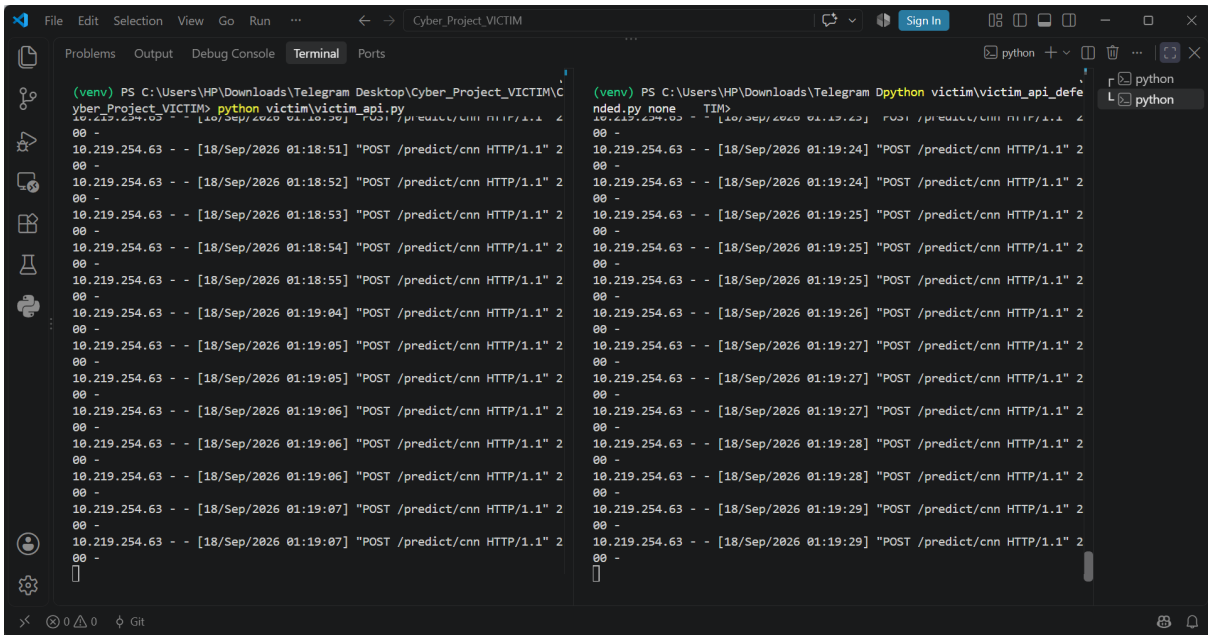


Figure 6: Victim PC: both Flask servers running and logging the incoming remote HTTP prediction requests from the attacker PC. Each access line carries the attacker's source IP, `POST /predict/linear` or `POST /predict/cnn`, and `200 OK`.

```
[victim_api] UNDEFENDED victim API on http://localhost:5000
  POST /predict/linear POST /predict/cnn GET /health
 * Running on http://0.0.0.0:5000
<attacker-ip> - - [.. ..] "GET /health HTTP/1.1" 200 -
<attacker-ip> - - [.. ..] "POST /predict/linear HTTP/1.1" 200 -
<attacker-ip> - - [.. ..] "POST /predict/cnn HTTP/1.1" 200 -
```

```
[victim_api_defended] DEFENDED victim API on http://localhost:5001
  fixed default policy: {'mode': 'none'}
 * Running on http://0.0.0.0:5001
<attacker-ip> - - [.. ..] "POST /predict/cnn HTTP/1.1" 200 -
```

The important observation from the victim's side is what is *not* there: the victim sees only well-formed prediction requests returning `200 OK`. Nothing in its logs distinguishes the 785 extraction queries from ordinary traffic, which is exactly why model extraction is hard to detect at the application layer.

6.2 Attacker PC (client)

```
Checking victim API servers ...      You, 58 minutes ago • added two device setup and at
  undefended server up at http://10.219.254.169:5000
  defended  server up at http://10.219.254.169:5001

Loading real MNIST test set for evaluation
[data_mnist] Trying shipped local data/MNIST/raw (offline IDX parser) ...
[data_mnist] Loaded shipped MNIST raw files (no network needed).
[data_mnist] Train set: (1, 1, 28, 28), Test set: (2000, 1, 28, 28)
victim_meta.json not found -> measuring victim accuracy over the API ...
victim_linear accuracy (measured): 0.9010
victim_cnn    accuracy (measured): 0.9650

======================================================================
Equation-Solving Attack (Tramer et al. 2016) -- over HTTP API
======================================================================
[equation-solving:no_defense] budget=  785  accuracy=0.9010  fidelity=1.0000
[equation-solving:no_defense] budget= 1570  accuracy=0.9010  fidelity=1.0000
[equation-solving:no_defense] budget= 3925  accuracy=0.9010  fidelity=1.0000
[equation-solving:defense_round2] budget=  785  accuracy=0.0885  fidelity=0.0955
[equation-solving:defense_round2] budget= 1570  accuracy=0.3010  fidelity=0.3150
[equation-solving:defense_round2] budget= 3925  accuracy=0.3295  fidelity=0.3490
[equation-solving:defense_topk3] budget=  785  accuracy=0.0595  fidelity=0.0620
[equation-solving:defense_topk3] budget= 1570  accuracy=0.3175  fidelity=0.3265
[equation-solving:defense_topk3] budget= 3925  accuracy=0.2890  fidelity=0.2975
[equation-solving:defense_labels_only] budget=  785  accuracy=0.0875  fidelity=0.0840
[equation-solving:defense_labels_only] budget= 1570  accuracy=0.2645  fidelity=0.2730
[equation-solving:defense_labels_only] budget= 3925  accuracy=0.2460  fidelity=0.2495
[equation-solving:defense_flip30] budget=  785  accuracy=0.1185  fidelity=0.1185
[equation-solving:defense_flip30] budget= 1570  accuracy=0.2590  fidelity=0.2070
[equation-solving:defense_flip30] budget= 3925  accuracy=0.2385  fidelity=0.1890

======================================================================
Knockoff Attack (Orekondy 2019 / Jagielski 2020) -- over HTTP API
======================================================================
[data_mnist] Trying shipped local data/MNIST/raw (offline IDX parser) ...
[data_mnist] Loaded shipped MNIST raw files (no network needed).
[data_mnist] Train set: (5000, 1, 28, 28), Test set: (1, 1, 28, 28)
[knockoff:closed:cnn] budget=  100  accuracy=0.1910  fidelity=0.1910
[knockoff:closed:cnn] budget=  300  accuracy=0.6680  fidelity=0.6765
[knockoff:closed:cnn] budget=  800  accuracy=0.8535  fidelity=0.8710
[knockoff:closed:cnn] budget= 2000  accuracy=0.9170  fidelity=0.9350
[knockoff:closed:cnn] budget= 4000  accuracy=0.9390  fidelity=0.9580
[knockoff:open:cnn] budget=  100  accuracy=0.1075  fidelity=0.1075
[knockoff:open:cnn] budget=  300  accuracy=0.1075  fidelity=0.1075
[knockoff:open:cnn] budget=  800  accuracy=0.1075  fidelity=0.1075
[knockoff:open:cnn] budget= 2000  accuracy=0.1075  fidelity=0.1075
[knockoff:open:cnn] budget= 4000  accuracy=0.1075  fidelity=0.1075
```

Figure 7: Attacker PC: `run_experiment.py` reaching both remote victim servers and reporting the extraction results as the query budget is swept.

The attacker's log confirms that both remote victim servers were reachable and records the main extraction results. A concise excerpt is shown below; the full logs for all six scripts are in `results/log_*.txt` and the raw numbers in `results/results.json`, `arch_transfer.json` and `open_world.json`.

```
Checking victim API servers ...
  undefended server up at http://10.219.254.169:5000
  defended server up at http://10.219.254.169:5001

Loading real MNIST test set for evaluation
victim_meta.json not found -> measuring victim accuracy over the API ...
victim_linear accuracy (measured): 0.9010
victim_cnn accuracy (measured): 0.9650

[equation-solving:no_defense] budget= 785 accuracy=0.9010 fidelity=1.0000

[knockoff:closed:cnn] budget= 4000 accuracy=0.9390 fidelity=0.9580
[knockoff:open:cnn] budget= 4000 accuracy=0.1075 fidelity=0.1075

Saved results/results.json
Done.
```

Note the third line of that log: the attacker has no `victim_meta.json` and no victim files at all, so it *measures* the victim's accuracy by querying the API. That is the black-box property made visible — the attacker folder contains no `victim/` code and no `.pth` weights.

6.3 Single Packet / Frame View (demo_query)

The API contract is the "packet" of this project. Below is one real MNIST image (a digit 6) sent to the CNN victim, shown undefended and then under each defense, copied from `results/log_demo_query.txt`.

```
POST /predict/cnn
Request : {"image": [[ 28x28 pixel floats in 0..1 ]]}

# UNDEFENDED -- http://10.219.254.169:5000/predict/cnn
Response: {"probabilities": [0.0172, 0.0, 0.0069, 0.0, 0.0001,
                             0.0, 0.9757, 0.0, 0.0001, 0.0],
          "label": 6, "defense": "none"}

# DEFENDED -- round to 2 decimals (port 5001)
Response: {"probabilities": [0.0198, 0.0, 0.0099, 0.0, 0.0,
                             0.0, 0.9703, 0.0, 0.0, 0.0],
          "label": 6, "defense": "round(2d)"}
# rounded AND renormalised to sum to 1 -- hence 0.0198, not 0.02

# DEFENDED -- top-3 truncation (port 5001)
Response: {"probabilities": [0.0172, 0.0, 0.0069, 0.0, 0.0,
                             0.0, 0.9759, 0.0, 0.0, 0.0],
          "label": 6, "defense": "top-3"}
# only the 3 largest scores survive; the rest are zeroed

# DEFENDED -- labels-only (port 5001)
Response: {"probabilities": [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
          "label": 6, "defense": "label_only"}

# DEFENDED -- flip(eps=0.3) (port 5001)
Response: {"probabilities": [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
          "label": 1, "defense": "flip(eps=0.3)"}
# this image was flipped 6 -> 1, and stays 1 on every re-query
```

Reading down the `label` field makes the whole defense story visible in a single frame: the first four responses all still say `6`, which is why they stop equation-solving but leave knockoff untouched; only the last one changes the label.

7 Defense Mechanism

7.1 The Four API-Layer Defenses

All defenses run server-side, at the API boundary (`server_core.py`), on the probability vector just before it is sent — the model itself is never changed, and the attacker can only ever observe the post-defense output.

- **Round:** rounds confidence scores to a set number of decimals.
- **Top-k truncation:** keeps only the top-k class scores.
- **Labels-only:** returns only the predicted label, no scores.
- **Flip:** with probability ϵ returns a wrong label. It is made deterministic per input (keyed by a BLAKE2b hash of the image plus a server-side secret), so the same image always gets the same answer.

7.2 Why Each Defense Does / Does Not Work (Table 1)

Each attack depends on a different field, so we measure each column on the model that attack actually targets: confidence distortion on the LINEAR victim (equation-solving), and label change on the CNN victim (knockoff).

Defense	**Conf. distortion $\lvert\Delta p\rvert$ (LINEAR / eq-solving)**	**Top-1 label change (CNN / knockoff)**
round(2 decimals)	0.0214	0.00%
top-3 truncation	0.1064	0.00%
labels-only	0.4110	0.00%
flip($\epsilon = 0.30$)	0.8719	30.40%

Table 7: Confidence-hiding defenses distort the linear victim's probabilities (so equation-solving dies) but do not change the CNN's top-1 label (so knockoff is untouched). Only flip moves the label-change column.

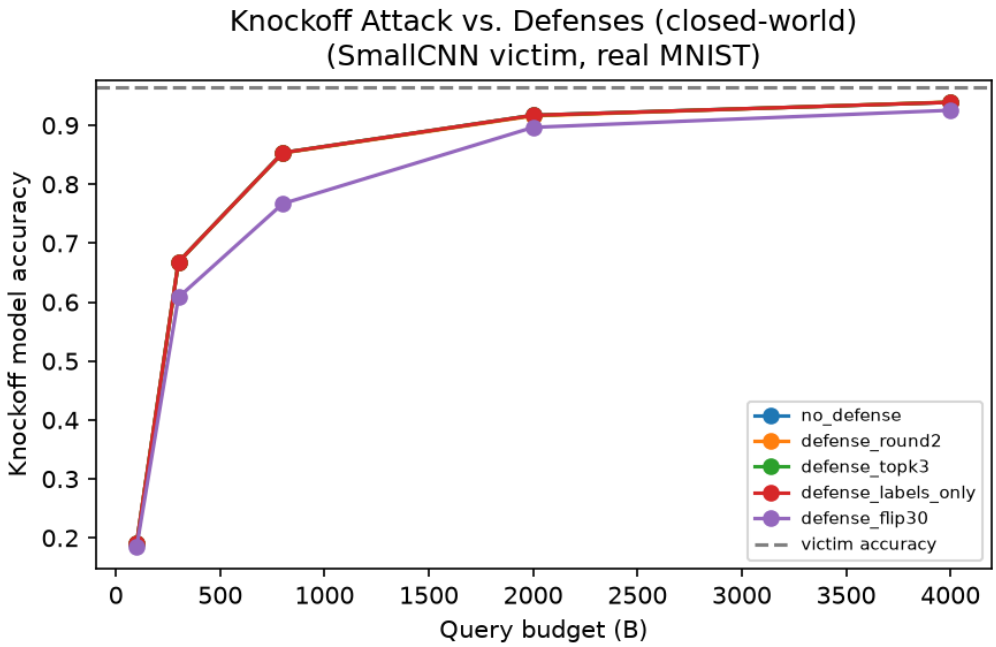


Figure 8: Closed-world knockoff accuracy versus query budget with no defense and under the four API-layer defenses. The round, top-3 and labels-only curves lie on top of the undefended one.

7.3 Flip Defense: Utility vs. Security Trade-off (Table 2)

ϵ	Honest acc.	Knockoff acc. (B=1500)	Knockoff fidelity
0.00	96.50%	92.40%	93.90%
0.10	86.05%	90.90%	82.40%
0.20	78.10%	89.25%	73.75%
0.30	67.55%	86.95%	62.40%
0.50	48.15%	83.85%	43.35%

Table 8: Label-flip defense: utility/security trade-off across flip rates, measured at a fixed query budget of $B = 1500$.

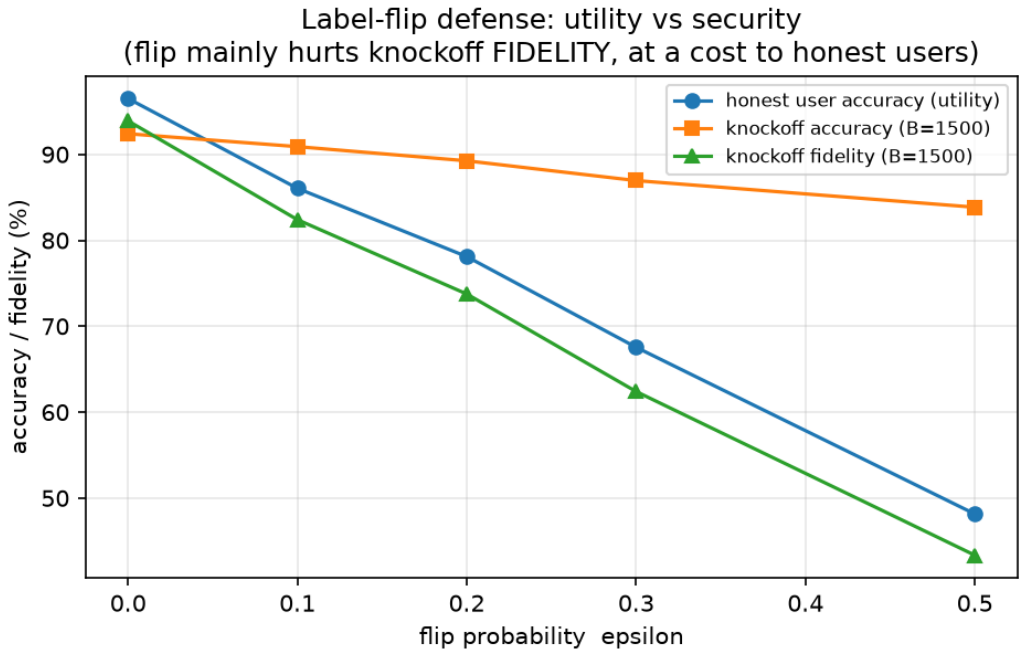


Figure 9: Utility/security trade-off of the label-flip defense.

7.4 Does the Defense Work as Expected?

Mostly yes, with two honest nuances. The confidence-hiding defenses (round / top-k / labels-only) work as intended against equation-solving, dropping its fidelity from 100% to 6–10% at the minimum budget, but they do nothing to knockoff, because they keep the top-1 label that knockoff relies on (Table 7: 0.00% label change, and the overlapping curves in Figure 8). The first nuance is the one already discussed in Section 4.4: against a patient attacker these defenses *degrade* equation-solving to a 25–35% plateau rather than eliminating it, so "fidelity collapses" is only true at the minimum budget.

The flip defense is the only one that touches knockoff, and here the second nuance appears. As ϵ grows, knockoff *fidelity* falls sharply (93.9% $\rightarrow$ 62.4% at $\epsilon = 0.3$), so flip does defeat an attacker who wants to copy the victim exactly. But knockoff *accuracy* barely drops (92.4% $\rightarrow$ 87.0%), because a CNN trained on many examples tolerates a minority of wrong labels — so an attacker who only wants a good model is mildly inconvenienced, not stopped. Meanwhile honest users lose accuracy of about ϵ (96.5% $\rightarrow$ 67.5%). This is the utility/security trade-off that all three papers name but none of them measures, and in the accuracy-versus-fidelity language of Jagielski et al. the conclusion is sharp: output perturbation defends confidentiality of the *function*, not of the *task*.

7.5 Determinism Proof (flip cannot be averaged away)

A naive random flip could be beaten by sending the same image many times and taking a majority vote. Our flip is deterministic per input, so this does not work. The proof (15 repeats, 500 images, $\epsilon = 0.3$):

Check	Result
Images whose label changed over 15 repeats	0 / 500
Single-query agreement with true label	71.6%
15-repeat majority agreement (deterministic)	71.6%
Gain from re-querying	+0.0%
Naive per-query-random flip, majority vote	100.0%

Table 9: Deterministic keying gives no gain from re-querying (71.6% $\rightarrow$ 71.6%), while a naive per-query random flip is fully averaged away (back to 100%).

8 Assumptions

- The attacker has query-only black-box access to the prediction API.
- The attacker does not have the victim's model weights.
- The attacker does not have the victim's original training data.
- Query budget is counted per image sent to the victim API.
- The secret used by the flip defense is not known to the attacker.
- The attacker has a small labelled test set to measure accuracy/fidelity.
- For equation-solving, the random query points are assumed independent enough to solve the linear system.
- The victim's output layer is log-linear for equation-solving to apply at all; the knockoff attack makes no such assumption.
- Network connectivity between the two PCs is available for the whole experiment, and the victim's IP is known to the attacker.

9 Conclusion

A query-only prediction API is inherently exposed to model extraction. A log-linear model can be copied exactly with simple linear algebra in as few as 785 queries. A non-linear CNN can be copied by learning from its own answers, reaching about 94% accuracy (roughly 0.97$\times$ the victim's own 96.5%) without knowing its architecture or data, as long as the query images are realistic. On the defense side, hiding confidence scores stops equation-solving at small budgets but only degrades it at larger ones, and does nothing at all to knockoff; only label flipping hurts knockoff — and even then it mainly hurts fidelity, at a direct cost to honest users. In short, no single defense is free or complete: security and prediction utility trade off against each other.

References

[1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing Machine Learning Models via Prediction APIs," *25th USENIX Security Symposium*, 2016.

[2] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff Nets: Stealing Functionality of Black-Box Models," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[3] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, "High Accuracy and High Fidelity Extraction of Neural Networks," *29th USENIX Security Symposium*, 2020.